

PROGETTO BIG DATA

“LINK PREDICTION IN UNA RETE SOCIALE CON GNN”

SANTO LO BIANCO

SOMMARIO

Introduzione *3*

Dataset e analisi esplorativa *4*

Rappresentazione della rete e preprocessing *6*

Controlli di integrità e coerenza 7

ARCHITETTURA DELLA GNN *7*

Baseline euristiche e metriche di valutazione *9*

Sezione sperimentale *10*

Attività con le baseline 10

Attività con GNN 11

Studio dell'ablazione e del degree bias 11

Risultati sperimentali e confronto *12*

Limiti metodologici, sperimentali e considerazioni etiche *14*

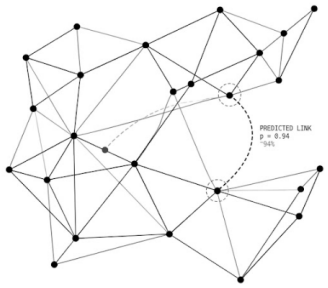
Limiti 14

Considerazioni etiche 14

INTRODUZIONE

Le reti complesse rappresentano uno strumento fondamentale per descrivere e analizzare sistemi caratterizzati da relazioni tra entità. Tra queste applicazioni rientrano:

- social network;
- reti di collaborazione scientifica;
- sistemi di interazione tra utenti.



Uno dei problemi di particolare interesse è la **Link Prediction**, ovvero il compito di stimare la **probabilità che tra due nodi esista una relazione ancora non osservata**.

L'obiettivo è quindi individuare collegamenti mancanti o potenzialmente futuri, sfruttando le informazioni strutturali già disponibili nel grafo.

Questo problema nasce dal fatto che un grafo osservato rappresenta soltanto una parte delle relazioni effettivamente esistenti o che potrebbero svilupparsi nel tempo.

La difficoltà consiste quindi nell'individuare quali coppie di nodi abbiano maggiori probabilità di essere collegate, basandosi sulle informazioni disponibili senza conoscere in anticipo le relazioni mancanti.

Lo scopo di questo progetto è studiare e confrontare differenti approcci alla Link Prediction, analizzando sia metodi euristici tradizionali e sia modelli basati su **Graph Neural Network(GNN)**.

In particolare, vengono considerate quattro baseline euristiche: **Common Neighbors**, **Jaccard Coefficient**, **Adamic-Adar** e **Preferential Attachment**.

Successivamente, viene implementato un modello basato su **GraphSAGE**, una GNN in grado di apprendere rappresentazioni dei nodi aggregando informazioni dai rispettivi vicini.

Le rappresentazioni apprese vengono utilizzate da un decoder per stimare la probabilità di esistenza di un collegamento tra due nodi.

La valutazione viene condotta attraverso un protocollo basato sulla suddivisione degli archi in insiemi di **training**, **validation** e **test**, accompagnato da un **campionamento controllato di archi negativi**(che non esistono nel grafo originale).

Le prestazioni vengono misurate attraverso: **ROC-AUC, Average Precision, Hits@K**.

Oltre al confronto delle prestazioni si fa: un'analisi di **ablation**, ossia vede come le singole feature influenzano il risultato e uno studio sul **degree bias** delle euristiche, verificando come le prestazioni varino in funzione alla popolarità degli archi.

DATASET E ANALISI ESPLORATIVA

Per valutare il comportamento dei modelli di link prediction su reti con caratteristiche differenti, sono stati selezionati tre dataset reali: **Facebook**, **ca-GrQc** e **ca-AstroPh**, scaricati dal sito del **Stanford Large Network Dataset Collection (SNAP)**.

I link sono i seguenti:

- Facebook ➔ https://snap.stanford.edu/data/ego-Facebook.html
- ca-GrQc ➔ https://snap.stanford.edu/data/ca-GrQc.html
- ca-AstroPh ➔ https://snap.stanford.edu/data/ca-AstroPh.html

I dataset rappresentano contesti differenti: Facebook descrive una **rete sociale**, mentre ca-GrQc e ca-AstroPh rappresentano **reti di collaborazione scientifica**, nelle quali i nodi corrispondono a ricercatori e gli archi indicano collaborazioni tra di essi. Tutti e tre i dataset sono stati rappresentati come **grafi non orientati**.

Per ciascuno è stata condotta un'analisi strutturale preliminare, considerando il numero di nodi e archi, la densità, il numero di componenti connesse e le principali statistiche relative alla distribuzione dei gradi.

I dataset presentano le seguenti strutture:

Dataset	Nodi	Archi	Densità	Grado medio	Grado mediano	Componente maggiore(%nodi)
Facebook	4039	88234	0,01	43,69	25	100%
ca-GrQc	5242	14496	0,001	5,53	3	79,32%
ca-AstroPh	18772	198.110	0,0011	21,11	9	95,37%

I tre dataset dimostrano strutture differenti questo è utile per valutare modelli in condizioni differenti: da una rete relativamente più densa e connessa come Facebook, a reti più sparse e frammentate come ca-GrQc o ca-AstroPh. Tali differenze permettono di influenzare la capacità dei metodi di individuare nuove connessioni.

Per approfondire la struttura delle reti e avere una fonte visiva su come sono organizzate le stesse, si sono selezionati 300 nodi con il grado maggiore e si è costruito il sottografo contenente esclusivamente gli archi esistenti dei nodi selezionati.

La disposizione dei nodi viene calcolata attraverso l'algoritmo spring layout di NetworkX, che utilizza un meccanismo di palline(nodi) e molle(archi), che le tira e spinge finché la rete non assume una forma chiara e bilanciata.

Questa visualizzazione consente di osservare la distribuzione delle connessioni tra i nodi maggiormente connessi e di individuare eventuali strutture caratterizzate da una forte concentrazione di collegamenti.

ca-AstroPh	ca- GrQc	Facebook



Nel caso di ca-AstroPh, il sotto-grafo presenta una **struttura centrale particolarmente densa e coesa**. I nodi selezionati risultano essere fortemente connessi. Questa osservazione è coerente con la presenza di una componente principale che comprende il 95,37% dei nodi e con un grado medio pari a 21,11.

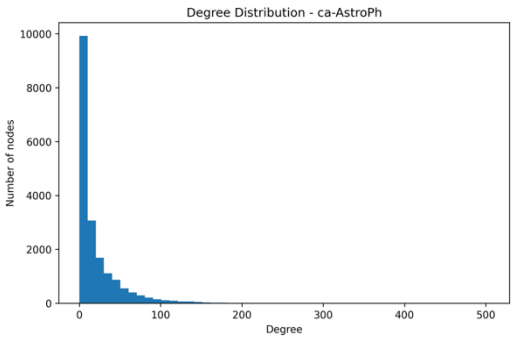
Il sottografo di ca-GrQc mostra una **configurazione più frammentata**, caratterizzata dalla presenza di un gruppo centrale e porzioni periferiche meno connesse tra loro. Ciò è coerente con le caratteristiche strutturali del grafo completo, il cui grado medio è di 5,53 e una componente principale contenente il 79,32%.

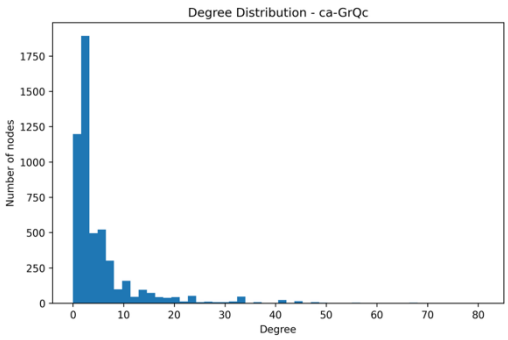
La rete Facebook presenta una configurazione visiva differente, nella quale i nodi selezionati sembrano organizzarsi in diversi gruppi. Tuttavia, è importante ricordare che il grafo originale è costituito da un'unica componente connessa.

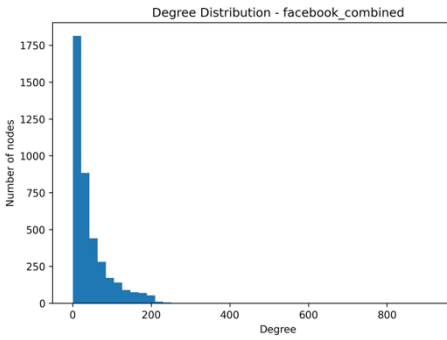
Gruppi apparentemente non connessi possono essere collegati da nodi che non sono presenti nel sotto-grafo.

Nel complesso, le visualizzazioni mettono in evidenza una diversa organizzazione dei nodi maggiormente connessi nei tre dataset.

Analizzando la distribuzione dei gradi dei nodi si nota che:







I tre istogrammi mostrano la distribuzione dei gradi per i rispettivi dataset, evidenziando in tutti i casi una tipica topologia a coda lunga, tipica delle **power law**.

Questa conformazione indica che la stragrande maggioranza dei nodi possiede pochissime connessioni, mentre una piccolissima frazione di nodi (gli Hub) risulta iper-connessa.

Nello specifico, l'estensione della coda varia significativamente in base al dominio: il grado massimo si ferma intorno a 80 nella rete ca-GrQc e arriva a circa 500 connessioni negli altri due dataset.

Rappresentazione della rete e preprocessing

I tre dataset utilizzati vengono rappresentati mediante grafi non orientati (G = (V, E) con V insieme dei nodi ed E insieme degli archi). Essi sono forniti sotto forma di **edge list**, ossia file dove ogni riga rappresenta un collegamento tra coppie di nodi. Il caricamento avviene attraverso la libreria **Network X**, utilizzata dal file **dataloader.py**.

Vengono **rimossi eventuali self-loop**, che ad esempio nelle reti di co-autoraggio rappresentano studi intrapresi da singole persone, e che non sono rilevanti per la link prediction.

Una volta caricato il grafo, vengono calcolare alcune statistiche strutturali utili a descriverne le caratteristiche principali tra cui: il numero di nodi e archi, la densità, il numero di componenti connesse e la distribuzione dei gradi. Al fine di valutare la capacità dei modelli di individuare collegamenti non osservati durante l'addestramento, gli archi vengono suddivisi in:

- **training** → insieme degli archi che il modello può utilizzare durante l'addestramento.
- **validation** → insieme degli archi che viene impiegato durante l'addestramento per controllare come sta funzionando. In particolare, questo insieme serve a confrontare le prestazioni della GNN, confrontare le epoche e selezionare il checkpoint migliore del modello.
- **test** → insieme degli archi utile alla valutazione finale.

Il procedimento è implementato in **split.py** che utilizza un **sedd fissato**, così da rendere **riproducibili le operazioni di mescolamento e suddivisione**.

Il grafo destinato all'addestramento viene ricostruito mantenendo i nodi originali, ma escludendo gli archi non appartenenti al training set.

Per l'addestramento della GNN viene effettuata un'ulteriore suddivisione degli archi appartenenti al training. In particolare, la maggior parte viene assegnata al **message passing**, mentre il restante costituisce l'insieme degli **archi supervisionati**, utilizzati per calcolare la **loss function**(utile a valutare il modello e aggiornare i pesi).

Il **message passing** è il **meccanismo che permette ad un nodo di raccogliere informazioni dai nodi con cui è collegato**, costruendo una rappresentazione che tiene conto del contesto in cui si trova il nodo nella rete. Ogni nodo riceve informazioni dai nodi adiacenti, le aggrega e aggiorna la propria rappresentazione.

Nella GNN ci sono due layer perché:

- **primo layer** → raccoglie le informazioni dei propri vicini diretti;
- **secondo layer** → incorpora informazioni sui nodi a distanza 2, attraverso i propri vicini.

Per affiancare agli esempi positivi degli esempi negativi, viene applicata una procedura di negative sampling, implementata dal file **negative_sampling.py**.

In questo file vengono generate coppie casuali di nodi non presenti nel grafo originale. Il numero di archi negativi generati per ciascun insieme è uguale al numero di positivi corrispondenti, ottenendo così un campionamento bilanciato.

La procedura, per un maggiore rigore, esclude:

- archi presenti nel grafo originale;
- coppie negative;
- archi presenti in altri insiemi.

La preparazione complessiva degli insiemi di dati e relativi archi negativi è coordinata dal modulo pipeline_data.py, che richiama le funzioni responsabili del tutto.

Per la valutazione mediante **Hits@K** vengono predisposte **specifiche query**, costituite da **un arco positivo e un insieme di candidati negativi**. In negative_samplig.py, i candidati negativi vengono selezionati escludendo i collegamenti effettivamente presenti nel grafo originale.

Questa configurazione permette di valutare la capacità del modello di assegnare un punteggio più elevato al collegamento positivo rispetto a un insieme più ampio di alternative.

Controlli di integrità e coerenza

Per verificare la correttezza della preparazione dei dati, la pipeline prevede una serie di controlli automatici, implementati in **src/utils/sanity_checks.py** e coordinati da **src/utils/pipeline_checks.py**.

Viene verificata la corretta separazione degli archi positivi tra training, validation e test, l'assenza di sovrapposizioni tra gli esempi negativi e la loro effettiva assenza dal grafo orginale. Inoltre, viene controllata la coerenza dello splitting utilizzato dalla GNN, verificando la separazione tra archi di message passing e archi supervisionati.

Una particolare attenzione è riservata ai nodi isolati, dato che dopo lo splitting del grafo alcuni nodi potrebbero rimanere isolati.

In caso di fallimento dei controlli di validità, viene sollevata un'eccezione che evita di proseguire con dati incoerenti.

ARCHITETTURA DELLA GNN

Per affrontare la link prediction è stata implementata una **Graph Neural Network** basata sull'architettura **GraphSAGE**(sample and aggregate), che **campiona casualmente un numero fisso di vicini per ciascun livello e utilizza una funzione di aggregazione per raccogliere le informazioni provenienti dai vicini**.

L'architettura segue una struttura **encoder-decoder**: **l'encoder elabora le informazioni strutturali e le caratteristiche dei nodi, producendo i relativi embedding; invece, il decoder utilizza tali rappresentazioni per stimare la probabilità che una coppia di nodi sia collegata.**

Prima dell'elaborazione, il grafo viene convertito nel formato data di PyTorch Geometric. A ciascun nodo viene associato un indice intero consecutivo e gli archi vengono rappresentati mediante una matrice. Poiché i grafi non sono orientati, ogni arco viene inserito in entrambe le direzioni. Questa conversione, insieme alla costruzione delle caratteristiche dei nodi, è implementata nella funzione **network_to_pyg()** del file **src/gnn/data.py**. Ciascun nodo è descritto da due caratteristiche strutturali:

- **il grado normalizzato** → si rapporta il grado del nodo al grado massimo del grafo;
- **coefficiente di clustering** → livello di interconnessione tra i suoi vicini.

L'**encoder** è **costituito da due layer SAGEConv**(livelli convoluzionali), che permettono di aggiornare progressivamente la rappresentazione di ciascun nodo aggregando informazioni proveniente dal proprio vicinato.

Il **primo layer** riceve le due caratteristiche iniziali e **produce una rappresentazione intermedia a 64 dimensioni**. Segue una funzione di attivazione **ReLU**(non lineare che permette di esprimere relazioni più complesse) e infine il **secondo layer** che trasforma la rappresentazione intermedia in un **embedding finale di 32 dimensioni**.

L'architettura è definita nella classe del modello presente in **src/gnn/model.py**.

Per valutare le coppie di nodi sono stati implementati e confrontati due decoder differenti:

- **decoder MLP(Multi-Layer Perceptron)** ➔ costruisce una rappresentazione a partire dai due embedding, **combinando il loro prodotto e applicando la differenza assoluta**. I due vettori vengono concatenati e forniti ad una rete neurale composta da 64 unità, seguito da RELU, e da un **layer finale che restituisce un singolo valore**.
- **decoder Dot Product** ➔ calcola il prodotto scalare tra gli embedding dei due nodi. La soluzione non introduce ulteriori parametri addestrabili e costituisce un approccio più semplice rispetto a MLP.

Entrambe i decoder restituiscono un valore denominato **logit**, utilizzato durante l'addestramento, che attraverso una **sigmoide** si **può convertire in uno score compreso tra 0 e 1**, utilizzato per la plausibilità del collegamento.

L'addestramento della GNN è finalizzato a ottimizzare i parametri dell'encoder e del decoder(nel caso del decoder MLP), affinché il modello impari a distinguere coppie di nodi collegate da quelle non collegate.

A questo scopo vengono utilizzati esempi positivi ed esempi negativi, con un numero sempre pari dei due insiemi. La preparazione delle coppie e delle relative etichette è implementata nella funzione **prepare_dges()** del file **src/gnn/data.py**, mentre la generazione degli esempi negativi è gestita da **src/negative_sampling.py** e coordinata da **src/pipeline_data.py**.

Durante ciascuna epoca, l'encoder elabora il grafo di message passing e calcola gli embedding dei nodi. Il decoder successivamente utilizza tali rappresentazioni per produrre i logits relativi alle coppie supervisionate.

Le predizioni poi vengono confrontate con le etichette effettive mediante la funzione di perdita **Binary Cross Entropy with Logits**, che combina due passaggi fondamentali:

- **Sigmoide** → assegna uno score tra 0 e 1;
- **Binary Cross Entropy** → misura quanto la probabilità predetta dal modello si discosta dal target reale.

La funzione di perdita viene minimizzata attraverso la **backpropagation**, che calcola i gradienti rispetto ai parametri del modello, e l'ottimizzatore Adam. **Adam** o **Adaptive Moment Estimation** è l'algoritmo di ottimizzazione più utilizzato e permette di aggiornare i pesi della rete durante la backpropagation.

Adam funziona nel seguente modo:

- **Momentum** ➔ se un peso va nella stessa direzione, lo aiuta a superare i minimi locali e a non bloccarsi.
- **RMSprop** ➔ se inizia ad oscillare mantiene la traiettoria stabile.

Queste operazioni sono implementate nel file **src/gnn/training.py**, in particolare nelle funzioni dedicate al calcolo della loss e all'addestramento della rete. Al termine di ogni epoca il modello viene valutato sul validation set, misurando le prestazioni mediante ROC-AUC e Average Precision.

La selezione del modello avviene sulla base della ROC-AUC: quando viene raggiunto un valore migliore, i pesi del decoder e dell'encoder vengono conservati. Inoltre, è presente una strategia di **early stopping**, con un patience che di base è 50 epoche.

Se la ROC-AUC non migliora per 50 epoche consecutive, l'addestramento viene interrotto. Al termine viene ripristinato il checkpoint corrispondente all'epoca migliore.

Per rendere più robusto il confronto tra i due encoder, gli **esperimenti principali vengono ripetuti utilizzando ci diversi seed**. I risultati poi vengono sintetizzati attraverso media e deviazione standard.

Il numero massimo di epoche previsto per gli esperimenti principali è pari a 500. Tale limite è stato definito anche per contenere il costo computazionale complessivo degli esperimenti, considerando che ciascuna configurazione viene eseguita su cinque diversi seed.

Nonostante questa limitazione, il decoder MLP ha ottenuto risultati ottimi nelle valutazioni sperimentali. L'utilizzo di hardware perfomante permette di aumentare le epoche e garantisce una convergenza anche imposta da early-stopping.

Baseline euristiche e metriche di valutazione

Per valutare l'efficacia dell'approccio basato su GNN, sono state implementate **quattro baseline tradizionali** per il problema della link prediction: Common Neighbors, Jaccard Coefficient, Adamic-Adar e Preferential Attachment.

Questi metodi **stimano la possibilità che due nodi siano collegati in futuro attraverso le caratteristiche strutturali della rete**. A differenza della GNN, non richiedono alcuna fase di addestramento e non apprendono rappresentazioni latenti dei nodi, ma assegnano un punteggio alla coppia candidata sulla base di regole statistiche. Le quattro baseline sono:

- **Common Neighbors** ➔ due nodi hanno maggiori probabilità di essere collegati se condividono molti vicini.

  $$CN(u, v) = |\Gamma(u) \cap \Gamma(v)|$$

  Il limite principale è che non considera quanto siano importanti o popolari i vicini comuni.
- **Jaccard Coefficient** ➔ non considera quanti vicini hanno due nodi, ma quanto questi vicini siano numerosi rispetto all'insieme complessivo dei loro vicini.

  $$J(u, v) = \frac{|\Gamma(u) \cap \Gamma(v)|}{|\Gamma(u) \cup \Gamma(v)|}$$
- **Adamic-Adar** ➔ non tutti i vicini comuni hanno la stessa importanza, un vicino condiviso da pochi nodi è più significativo di un vicino collegato a moltissimi nodi.

  $$AA(u, v) = \sum_{z \in \Gamma(u) \cap \Gamma(v)} \frac{1}{log(degree(z))}$$

  Il limite è che richiede il calcolo dei gradi dei vicini comuni ma permette di valorizzare collegamenti più informativi.

- **Preferential Attachment** ➔ questo algoritmo si basa sull'idea che i nodi popolari hanno una maggiore probabilità di acquisire nuove relazioni.

  $$PA(u, v) = d(u) \cdot d(v)$$

L'implementazione delle quattro baseline è contenuta nel modulo **src/baseline.py** e utilizza NetworkX per il calcolo dei punteggi. Tutti i metodi ricevono in ingresso il grafo di riferimento e un insieme di coppie di nodi da valutare, restituendo un punteggio per ciascuna coppia.

Per confrontare le prestazioni dei diversi approcci sono state utilizzate tre metriche: **ROC-AUC, Average-Precision e Hits@K**. **Le prime due valutano la capacità di distinguere globalmente gli archi positivi da quelli negativi, mentre Hits@K analizza la capacità del modello di collocare un collegamento corretto nelle prime posizioni in graduatoria.**

La **ROC-AUC**(Area Under the Receiver Operating Characteristic Curve) **misura la capacità del modello di assegnare punteggi maggiori agli archi esistenti rispetto agli archi inesistenti**. Può essere interpretata come la probabilità che un arco positivo scelto riceva uno score superiore a quello di un arco negativo scelto casualmente.

Un valore pari a 1 indica una separazione perfetta tra due classi, mentre un valore pari a 0.5 corrisponde a una capacità equivalente alla scelta casuale.

L'**Average Precision sintetizza le prestazioni lungo la curva Precision-Recall**, ossia il rapporto tra archi positivi realmente positivi e alla capacità di raccolta di tutti gli archi positivi. È particolarmente utile quando si vuole analizzare la capacità del modello di individuare archi positivi.

Hits@K valuta la capacità del modello di posizionare un arco positivo tra i primi K candidati di una lista. Una singola query positiva a fronte di un insieme di candidati negativi. Si determina quindi la posizione dell'arco in graduatoria e si considera un'hit quando il rango è minore o uguale a K.

Inoltre, nell'implementazione c'è una gestione dei punteggi a pari merito. In particolare, viene adottata la modalità pessimistica, che penalizza l'arco positivo attribuendogli un rango successivo rispetto a candidati negativi con lo stesso punteggio.

Le metriche sono implementate nel modulo **src/evaluation.py**. La funzione **evaluate_scores()** calcola ROC-AUC e Average Precision a partire dai punteggi positivi e negativi, mentre **hits_at_k()** calcola la metrica di ranking.

SEZIONE SPERIMENTALE

La cartella **experiments** raccoglie le procedure utilizzate per eseguire e confrontare gli esperimenti del progetto. Le attività sperimentali sono organizzate in più fasi:

- valutazione delle baseline;
- addestramento e valutazione della GNN;
- confronto tra decoder;
- analisi di ablazione delle feature;
- studio della dipendenza delle prestazioni dalla popolarità dei nodi.

Tutti gli esperimenti sono stati condotti sui tre dataset sopraccitati, eseguendo la stessa procedura generale.

Le baseline euristiche sono state valutate utilizzando due diverse configurazioni del grafo, per verificare quanto la quantità di informazioni possa influenzare le prestazioni.

Attività con le baseline

Nel primo scenario, le euristiche operano sul grafo di training, mettendo a disposizione una maggiore quantità di informazioni topologiche.

Nel secondo scenario, le euristiche vengono applicate al grafo utilizzato dalla GNN durante la fase di message passing, dove sono rimossi anche gli archi positivi riservati alla supervisione del modello.

Il confronto tra i due scenari permette di osservare l'effetto della riduzione delle informazioni topologiche sulle prestazioni dei metodi euristici. Ovviamente, al fine di garantire un confronto coerente sono stati mantenuti stessi archi positivi e negativi di test e lo stesso insieme di query per il calcolo di Hits@K.

L'esecuzione della valutazione è coordinata dal modulo **experiments/baseline_evaluation.py**, attraverso la funzione **evaluate_baselines()**. Questa applica ogni metodo agli archi positivi e negativi di test, calcola ROC-AUC e Average Precision e richiama la procedura dedicata al calcolo di Hits@K(valuate_baseline_hits()).

Inoltre, l'**intero processo viene coordinato da run_baseline_experiments()** che esegue valutazione sia su train graph che su message-passing graph.

ATTIVITÀ CON GNN

La seconda parte della sperimentazione riguarda il modello basato su GNN, consentendo di osservare se la rete riesce a prevedere i collegamenti e ad essere competitiva rispetto ai metodi euristici.

Come citato nelle precedenti sezioni, si utilizza un encoder tale GraphSAGE e due decoder diversi: MPL e Dot Product. Le prestazioni finali vengono misurate sul test set, utile a calcolare ROC-AUC, Average Precision e Hits@K.

Per ridurre l'influenza della stocasticità associata ai pesi e all'addestramento, gli esprimenti sono stati ripetuti utilizzando 5 seed differenti(1, 2, 3, 4, 5). Questa scelta permette di valutare le prestazioni del modello ma anche stabilità tra esecuzioni differenti.

Questa procedura è implementata dalla funzione **run_gnn_multiseed()**, che ripete l'addestramento e la valutazione per ciascun seed.

STUDIO DELL'ABLAZIONE E DEL DEGREE BIAS

Al fine di comprendere il contributo delle informazioni iniziali fornite alla GNN, è stato realizzato uno studio di ablazione sulle feature dei nodi. L'esperimento confronta quattro diverse modalità di rappresentazione:

- full: utilizza entrambe le feature previste dalla configurazione principale, ossia grado normalizzato e coefficiente di clustering.
- degree: utilizza esclusivamente il grado normalizzato del nodo.
- clustering: utilizza esclusivamente il coefficiente di clustering.
- constant: assegna a tutti i nodi una feature costante, eliminando le informazioni esplicite contenute nelle feature iniziali.

L'ablation study è implementato nel modulo **experiments/run_ablation.py**, attraverso la funzione **run_feature_ablation()**.

Un'ulteriore analisi sperimentale è dedicata allo studio del degree bias, ossia della dipendenza delle prestazioni delle baseline dal grado dei nodi coinvolti nelle coppie candidate. I metodi euristici possono favorire certi nodi connessi come nel Preferential Attachment.

Per approfondire questo aspetto gli archi positivi e negativi del test set vengono suddivisi in quattro quartili, sulla base del grado medio dei due estremi(nodi coinvolti dall'arco) dell'arco. Il grado dei nodi viene calcolato utilizzando il train graph, così da misurare la popolarità dei nodi rispetto alla struttura disponibile durante la fase di training.

All'interno di ciascun quartile vengono valutate separatamente le prestazioni delle quattro baseline.

L'analisi è implementata nel modulo **experiments/run_degree_bias.py**. La funzione **run_degree_bias_experiment()** coordina l'esecuzione dell'esperimento per tutte le baseline, mentre le funzioni **compute_edge_degree()**, **assign_degree_quartiles()** ed **evaluate_degree_bias()** si occupano rispettivamente del calcolo della popolarità degli archi, della loro assegnazione ai quartili e della valutazione delle metriche all'interno di ciascun gruppo.

Infine, i risultati degli esperimenti vengono conservati in file CSV, in modo da rendere possibile analisi e confronto tra dataset. L'esportazione è gestita dal modulo condiviso **experiments/export_results.py**, che mette a disposizione la funzione **export_rows()**.

RISULTATI SPERIMENTALI E CONFRONTO

In questa sezione vengono presentati e analizzati i risultati ottenuti durante la fase sperimentale con l'obiettivo di confrontare le prestazioni delle baseline euristiche e dei modelli basati su GNN.

Per ciascuno dei dataset menzionati prima vengono esaminati: risultati baseline, prestazioni della GNN con diversi decoder, contributo delle feature iniziali attraverso ablation study e dipendenza delle euristiche dal grado dei nodi.

La prima analisi riguarda le prestazioni delle quattro euristiche sui tre dataset considerati. I metodi sono stati valutati utilizzando le due configurazioni del grafo.

La tabella contiene i risultati ottenuti con Adamic-Adar, dato che presenta prestazioni migliori:

Dataset	Configurazione	ROC-AUC	AP	Hits@10
Facebook	Train Graph	0,9924	0,9914	0,819
Facebook	Message-Passing Graph	0,9904	0,9896	0,814
ca-GrQc	Train Graph	0,8953	0,8955	0,785
ca-GrQc	Message-Passing Graph	0,8700	0,8702	0,734
ca-AstroPh	Train Graph	0,9833	0,9835	0,905
ca-AstroPh	Message-Passing Graph	0,9756	0,9759	0,882

I risultati mostrano che Facebook presenta le prestazioni più elevate, seguita da ca-AstroPh. Mentre ca-GrQC presenta valori inferiori, soprattutto quando le euristiche vengono applicate al Message-Passing Graph.

Un comportamento differente si osserva per **Preferential Attachment, che ottiene risultati sensibilmente inferiori rispetto alle altre tre euristiche**.

I valori di ROC-AUC sul Message-Passing Graph sono pari a circa 0,83 su Facebook, 0,74 su ca-GrQc e 0,86 su ca-AstroPh. Anche le metriche di ranking confermano le difficoltà di questo metodo nel selezionare correttamente i collegamenti positivi tra i candidati.

Common Neighbors, Jaccard e Adamic-Adar mostrano invece prestazioni generalmente vicine tra loro. Questo suggerisce che, nei dataset analizzati, l'informazione relativa ai vicini comuni costituisce un segnale particolarmente utile per individuare possibili collegamenti.

Per valutare la GNN sono state eseguite cinque ripetizioni con seed differenti. La tabella seguente riassume le prestazioni dei due decoder:

Dataset	Decoder	ROC-AUC	AP	Hits@10	Hits@50	Hits@100
Facebook	MLP	0,9811 ± 0,0007	0,9804 ± 0,0006	0,5572	0,8512	0,9034
Facebook	Dot Product	0,8616 ± 0,0666	0,8505 ± 0,0805	0,1706	0,3746	0,5132
ca-GrQc	MLP	0,9016 ± 0,0018	0,9083 ± 0,0022	0,3438	0,5082	0,6156
ca-GrQc	Dot Product	0,8245 ± 0,0408	0,8414 ± 0,0382	0,1276	0,3482	0,4898
ca-AstroPh	MLP	0,9480 ± 0,0003	0,9487 ± 0,0003	0,2310	0,4954	0,6498
ca-AstroPh	Dot Product	0,9023 ± 0,0023	0,9085 ± 0,0021	0,1082	0,3798	0,5548

Il **decoder MLP ottiene prestazioni superiori al Dot Product** su tutti e tre i dataset e per tutte le metriche considerate. La differenza è particolarmente evidente su Facebook, dove il decoder MLP raggiunge una ROC-AUC media di 0,9811, contro 0,8616 del Dot Product. Anche su ca-GrQc e ca-AstroPh il vantaggio del decoder MLP è consistente.

Il decoder MLP presenta deviazioni standard molto contenute, indicando che le prestazioni rimangono simili al variare del seed.

Il Dot Product mostra invece una maggiore variabilità, soprattutto su Facebook e ca-GrQc. In particolare, su Facebook la deviazione standard della ROC-AUC raggiunge 0,0666, mentre quella di Hits@100 è pari a 0,1716.

Questi risultati evidenziano che **la scelta del decoder influenza sensibilmente la capacità predittiva del modello**.

Confrontando baseline e GNN si nota che **le baseline risultano particolarmente competitive sulle reti Facebook e ca-AstroPh, mentre la GNN MLP mostra un vantaggio su ca-GrQc in termini di ROC-AUC e Average Precision**.

L'**ablation study** è stato utilizzato per analizzare il contributo delle feature iniziali alla capacità predittiva della GNN. Sono state confrontate quattro configurazioni: **Full**, **Degree**, **Clustering** e **Constant**.

La **configurazione Full ottiene i risultati migliori su tutti e tre i dataset**. Questo indica che l'utilizzo combinato del grado normalizzato e del coefficiente di clustering fornisce informazioni utili alla previsione degli archi.

Considerando le feature singolarmente, la **configurazione Degree ottiene risultati superiori alla configurazione Clustering in tutti e tre i dataset**. Il grado normalizzato sembra quindi rappresentare un'informazione particolarmente utile per il modello, pur non essendo sufficiente a raggiungere le prestazioni della configurazione completa.

La **configurazione Constant presenta invece un calo significativo delle prestazioni**. Su Facebook la ROC-AUC scende a 0,5712, su ca-GrQc a 0,6990 e su ca-AstroPh a 0,4381. Questi valori mostrano che la presenza di feature iniziali informative contribuisce in maniera rilevante alla qualità delle rappresentazioni apprese.

I risultati particolarmente bassi ottenuti in questa configurazione suggeriscono che, **nelle condizioni sperimentali adottate, la sola struttura disponibile attraverso il message passing non è sufficiente a recuperare le prestazioni ottenute con feature informative**.

L'ultima **analisi sperimentale** riguarda la dipendenza **delle prestazioni delle baseline dal grado dei nodi coinvolti nelle coppie candidate**. I risultati mostrano una relazione generalmente positiva tra il grado dei nodi e le prestazioni delle baseline, soprattutto per ca-GrQc e ca-AstroPh.

Su ca-AstroPh, Common Neighbors passa da una ROC-AUC di 0,9200 nel primo quartile a 0,9922 nel quarto. Un andamento analogo si osserva per Jaccard e Adamic-Adar, che raggiungono rispettivamente 0,9922 e 0,9936 nel quarto quartile.

Su ca-GrQc la differenza tra i quartili è ancora più evidente. Common Neighbors passa da 0,7335 nel primo quartile a 0,9807 nel quarto, mentre Adamic-Adar passa da 0,7335 a 0,9809. **Le prestazioni delle euristiche risultano quindi sensibilmente inferiori per le coppie associate a nodi con grado più basso.**

Il **Preferential Attachment mostra risultati inferiori rispetto alle altre euristiche in tutti i quartili**, ma presenta comunque un miglioramento al crescere del grado dei nodi. Per esempio, su ca-AstroPh la ROC-AUC passa da 0,5985 nel primo quartile a 0,8155 nel quarto. Su ca-GrQc passa da 0,3872 a 0,8780, mentre su Facebook passa da 0,6524 a 0,8302.

Questi risultati sono coerenti con il funzionamento dei metodi considerati. Le euristiche basate sui vicini comuni tendono a beneficiare della presenza di nodi più connessi, che possono condividere un numero maggiore di vicini. Anche Preferential Attachment è direttamente legato al grado degli estremi della coppia e mostra quindi una dipendenza particolarmente evidente da questa caratteristica.

L'analisi evidenzia tuttavia che l'effetto non è identico per tutte le reti e per tutti i metodi. In particolare, su Facebook le prestazioni di Common Neighbors, Jaccard e Adamic-Adar raggiungono il valore massimo nel terzo quartile, mentre nel quarto si osserva una lieve diminuzione della ROC-AUC.

Nel complesso, l'esperimento mostra che le prestazioni delle baseline sono influenzate dalla popolarità strutturale dei nodi. Questo aspetto è particolarmente evidente su ca-GrQc e ca-AstroPh e rappresenta un elemento da considerare nell'interpretazione dei risultati della link prediction.

Limiti metodologici, sperimentali e considerazioni etiche

Limiti

Nonostante i risultati ottenuti, il progetto presenta alcuni limiti metodologici e sperimentali che è importante considerare per interpretare correttamente le prestazioni dei modelli e le conclusioni.

Sebbene i dataset presentino caratteristiche strutturali differenti, i risultati non possono essere automaticamente generalizzati a qualsiasi tipologia di rete. Le prestazioni dei modelli dipendono infatti dalla distribuzione dei gradi(come si è visto nel degree bias), dalla densità e dalla struttura delle connessioni.

L'analisi fornisce quindi indicazioni sui dataset considerati, ma non dimostra che gli stessi comportamenti si verifichino in reti appartenenti ad altri domini.

Un aspetto particolarmente rilevante riguarda la costruzione degli esempi negativi. Gli archi non presenti vengono usati come relazioni inesistenti, ma l'assenza non implica che la relazione non esista nella realtà. Questa approssimazione è comune negli esperimenti di link prediction, ma introduce una possibile imprecisione nella definizione del problema e nella valutazione dei modelli.

La rappresentazione dei nodi utilizzata nella GNN si basa su due caratteristiche strutturali: grado normalizzato e coefficiente di clustering, ossia attributi non esterni al grafo. L'esperimento di ablation ha permesso di valutare il contributo delle singole feature, ma non consente di stabilire quale sarebbe il comportamento del modello utilizzando rappresentazioni più ricche.

La sperimentazione con reti neurali si concentra sull'architettura GraphSAGE, confrontando due modalità di decodifica: MLP e dot product. Questa scelta consente di analizzare il contributo della componente di decodifica mantenendo costante l'encoder, ma non permette di confrontare il comportamento di GraphSAGE con quello di altre architetture GNN.

Infine, l'analisi del degree bias ha evidenziato una relazione tra il grado medio degli estremi degli archi e le prestazioni delle euristiche. Tuttavia, questa analisi descrive un'associazione osservata nei dataset e non dimostra un rapporto causale. Inoltre, la suddivisione in quartili e la diversa distribuzione dei gradi nelle tre reti rendono necessario interpretare i risultati tenendo conto delle caratteristiche specifiche di ciascun grafo.

Considerazioni etiche

La link prediction, oltre a rappresentare un problema di interesse tecnico e scientifico, solleva importanti questioni etiche legate alla privacy, alla protezione dei dati personali e al possibile utilizzo delle informazioni inferite.

Un modello potrebbe individuare una relazione potenziale tra due utenti sulla base delle loro connessioni comuni o delle caratteristiche strutturali dei rispettivi nodi.
Anche quando queste previsioni sono statisticamente plausibili, non è possibile considerarle automaticamente vere. La previsione di un collegamento non costituisce una prova dell'esistenza di una relazione.

In determinati contesti, la struttura di una rete può rivelare informazioni che gli individui non hanno scelto di condividere direttamente.
La previsione di nuove connessioni potrebbe contribuire a dedurre interessi, appartenenze o relazioni personali, con possibili conseguenze sulla riservatezza degli utenti.
Il rischio aumenta quando le previsioni vengono combinate con altre fonti di dati o utilizzate per ricostruire aspetti non immediatamente visibili delle relazioni sociali.

L'analisi del degree bias dimostra che le euristiche basate sulla struttura del grafo possono favorire nodi con un numero elevato di connessioni. ‘È quindi importante non interpretare le previsioni come valutazioni neutrali o prive di distorsioni.

Le previsioni generate da un modello dovrebbero essere considerate indicazioni probabilistiche e non informazioni certe. Un loro utilizzo automatico in contesti che riguardano persone reali potrebbe produrre conseguenze indesiderate, soprattutto quando le relazioni inferite vengono impiegate per prendere decisioni o formulare giudizi sugli individui.
Quindi, è necessario sempre un controllo umano perché non sono informazioni certe.

L'utilizzo di reti sociali reali richiede particolare attenzione alle modalità di raccolta, conservazione e trattamento dei dati. Si devono perciò limitare le informazioni utilizzate a quelle strettamente necessarie per l'obiettivo della ricerca, adottare misure di sicurezza adeguate e, quando possibile, utilizzare dati anonimizzati o aggregati.

In conclusione, la link prediction offre strumenti utili per analizzare e comprendere la struttura delle reti, ma la sua applicazione a contesti sociali richiede un approccio responsabile e sensibile. Le relazioni previste devono essere interpretate con cautela, considerando i limiti dei dati, le possibili distorsioni dei modelli e le implicazioni per la riservatezza delle persone coinvolte.